# Genetic Programming with Algebraic Simplification for Dynamic Job Shop Scheduling

Sai Panda
School of Engineering and Computer Science
Victoria University of Wellington
Kelburn 6011, New Zealand
Email: pandasai@myvuw.ac.nz

Yi Mei
School of Engineering and Computer Science
Victoria University of Wellington
Kelburn 6011, New Zealand
Email: yi.mei@ecs.vuw.ac.nz

***Abstract*—Genetic Programming (GP) has been successfully applied to evolve effective dispatching rules for Dynamic Job Shop Scheduling (DJSS). However, the GP-evolved rules are usually too complex and hard to interpret. In this paper, we aim to address this issue by evolving simpler rules without losing effectiveness. To this end, we develop a set of algebraic simplification operators based on our domain knowledge about dynamic scheduling, which can recursively convert a rule into a mathematically equivalent but simpler one. The algebraic simplification operators can guarantee that the individual stays equivalent before and after the simplification. Then, we develop a GP algorithm with these simplification operators. We compared the GP with the simplification operators with the baseline GP without simplification on a range of scheduling instances, and the results showed that using the algebraic simplification can slightly reduce the program size without sacrificing the test performance of the evolved dispatching rules. Furthermore, through deep analysis, we have also discovered the limitations of the pure algebraic simplification for GP to evolve DJSS dispatching rules, which can hardly simplify the individuals after the first generation.**

## I. Introduction

Scheduling pertains to the "allocation of scarce manufacturing resources to tasks over time" whose goal is to "determine when a job should be processed and which machine is used to process that job" [1]. Job Shop Scheduling (JSS) seeks to schedule jobs arriving at a shop floor while optimising specific objectives (e.g. trying to minimise the mean tardiness) [2] subject to some constraints such as the resource constraint and precedence constraint. JSS has many real-world applications, such as manufacturing [3] [4] and factory production scheduling [5].

In many real-world situations, new jobs arrive over time during the execution process of the schedule. No information about these jobs will be available until the exact moment they arrive. This type of situation is referred to as the Dynamic Job Shop Scheduling (DJSS). Mathematical programming [6] and other traditional optimisation methods are difficult to apply to DJSS. As new jobs randomly arrive into the shop floor, it is difficult to correctly and effectively adjust the current schedule that has been set to take into account the newly arrived jobs along with the existing jobs in the remaining schedule. Traditional solution optimisation approaches such as genetic algorithms typically require a time-consuming re-optimisation process for the remaining schedule, which cannot react in time.

The most commonly used approach for solving DJSS problems is to utilise dispatching rules. A dispatching rule is represented by a priority function. During the schedule execution process, whenever a machine becomes idle (i.e., completes the current operation), if its queue is not empty, then the dispatching rule calculates the priority of each operation in the queue, and selects the top priority operation for the idle machine to process next [1]. Some commonly used dispatching rules include Shortest Processing Time (selecting the least time-consuming operation) and Earliest Due Date (selecting the operation with the earliest due date).

In practice, it is difficult to manually designing effective dispatching rules, since there are various factors that need to be considered. These factors include the objective to be optimised (e.g., mean tardiness or maximum flowtime), utilisation level of the job shop, and due date factors [7]. To address this issue, Genetic Programming (GP) has been successfully used as an effective method for automatically designing effective dispatching rules for a given DJSS scenarios [8], [1].

While utilising GP can be an effective method of evolving dispatching rules for DJSS, the learned rules are often very large and complex. Hence, their interpretability is low as a result. Being interpretable is an important characteristic that a model (or a dispatching rule in our case) should ideally have. For example, we need to have a firm understanding about how the dispatching rule makes decisions during the scheduling process, so that we may improve it. Furthermore, the end-user cannot fully trust a solution if they do not understand its inner mechanisms.

One main reason as to why GP produces large and complex rules is that GP tends to evolve trees that contain redundant information/branches. This redundancy adds extra unnecessary nodes into the GP trees. Although the redundant branches can protect the useful building blocks from being destroyed during the evolutionary process, it can greatly affect the interpretability of the final rule evolved by GP. If we can remove these redundant nodes from the GP tree, we can decrease the overall rule size, thus improving its interpretability without sacrificing

978-1-7281-8393-0/21/$31.00 ©2021 IEEE

the effectiveness. Nevertheless, directly simplifying the final rule is usually ineffective due to the complex interactions between the nodes.

To address this issue, several methods to remove these redundancies during the evolutionary process have been created [9] [10] [11] [12]. These methods propose algebraic and numerical simplification operators to remove redundancies from the GP trees. Then the GP selects some trees in its population to be simplified during the search process. The GP with simplification operators were tested on classification and regression problems, and the results showed the effectiveness of the proposed simplification operators in obtaining both good and simple models.

In this paper, we extend the set of existing simplification operators, and propose a new set of problem-specific simplification operators for dispatching rules for DJSS. Specifically, in addition to the generic algebraic simplification operators proposed in [9] [10] [11] [12], we propose some extra ones that take into account the information about the JSS features (e.g. the range and sign of each feature used in the terminal set), to simplify the dispatching rules more effectively.

The goal of this paper is to develop a new GP with simplification operators for evolving both high-quality and simple dispatching rules for DJSS. To achieve this goal, we have the following research objectives:

1) Extend the existing simplification operators, and develop a new set of problem-specific simplification operators for DJSS that can take into account the characteristics of the problem features.
2) Develop a new GP algorithm that simplifies the rules by the proposed simplification operators during the evolutionary process.
3) Examine the effectiveness of the proposed GP with simplification for DJSS, and analyse the evolved rules by the newly proposed GP.

The rest of the paper is organised as follows. Section II describes the background, including the problem description and the related work. Section III depicts the newly proposed simplification operators that makes use of the problem specific information of DJSS. Section IV carries out the experimental studies, including the experiment design and results and discussions. Section V provides the conclusions and possible future directions.

## II. Background

### A. Job Shop Scheduling

In JSS, a set of jobs $\mathcal{J} = \{J_1, J_2, \ldots, J_n\}$ is to be processed by a number of machines $\mathcal{M} = \{M_1, M_2, \ldots, M_m\}$. Each job $J_i$ consists of a sequence of operations $J_i = (O_{i1}, O_{i2}, \ldots, O_{il_i})$, where $l_i$ is the number of operations of $J_i$. Each operation $O_{ij}$ can only be processed by its designated machine $m_{ij} \in \mathcal{M}$, and its processing time is $p_{ij}$. Each job $J_i$ has an arrival time $a_i$, a due date $d_i$, and a weight $w_i$, indicating the importance of completing the job by its due date (or the penalty for each unit of delay). The JSS problem is to design a schedule (i.e., the starting and completion time of each operation) to complete the jobs with the machines subject to the following constraints.

- (*Precedence Constraint*) Each operation $O_{ij}$ cannot be started until its precedent operation $O_{i(j-1)}$ has been completed.
- (*Resource Constraint*) Each machine can process at most one operation at a time.

JSS can have a variety of objective to be optimised. Below are the three examples objective functions that we will use in this study.

$$T_{mean} = \frac{1}{n}\sum_{i=1}^{n} \max\{C_i - d_i, 0\}, \tag{1}$$

$$T_{max} = \max_{i=1}^{n}\{\max\{C_i - d_i, 0\}\}, \tag{2}$$

$$MWT = \frac{1}{n}\sum_{i=1}^{n} w_i \max\{C_i - d_i, 0\}, \tag{3}$$

where $C_i$ is the completion time of the job $J_i$. Eq. (1) indicates the mean tardiness, Eq. (2) stands for the max tardiness, and Eq. (3) is the mean weighted tardiness.

### B. Related Work

*1) Existing Methods for JSS and DJSS:* Due to the NP-hardness of JSS, the exact methods [13] [14] for JSS, which aim to find the optimal solution, are typically applicable to only small scale problem instances. For more practical and larger problems, heuristic solution optimisation (meta-heuristic) methods such as simulated annealing [15] [16], tabu search [17] and genetic algorithms [18] [19] are more commonly used. Although the meta-heuristic methods cannot guarantee optimally, they usually can obtain a reasonably good solution in a given limited time budget, which are more practically applicable.

Under the dynamic environment such as unpredicted job arrivals, the traditional solution optimisation methods normally cannot react online effectively. As a reactive scheduling approach, using dispatching rules is promising for DJSS, since the dispatching rules can make real-time decisions effectively. There have been extensive studies for designing and analysing dispatching rules for both static and dynamic JSS [20] [21] [22] [23].

*2) Genetic Programming Hyper-Heuristics:* Due to its flexible representation (trees or graphs), GP has been successfully applied as a hyper-heuristic approach [24] [25], to automatically generate effective high-level heuristics (e.g., dispatching rules) for various complex decision making problems. In the context of DJSS, there have been extensive studies that uses GP to evolve dispatching rules [8] [1]. A number of design issues have been investigated and addressed, such as feature selection [26] [27], efficient evaluation using surrogates [28] [29] [30] and multitask GP [31] [32].

In addition to the scheduling domain, GP has also been applied to evolve heuristics for other problems, such as

knapsack/packing [33], vehicle routing [34], timetabling [35] and cloud resource allocation [36] [37]. The key idea is to determine a heuristic template, and use GP to evolve the criteria (typically a priority function) for some critical decision making steps (e.g. which customer to serve next in vehicle routing, and which bin to be placed for the next item in knapsack).

*3) Simplification Techniques for GP:* The existing simplification techniques for GP can be mainly divided into two categories: algebraic simplification and numerical simplification. The algebraic simplification techniques typically use the algebraic rules (e.g., $A - A = 0$, $A/A = 1$) to simplify the GP trees. The numerical simplification, on the other hand, consider the numeric characteristics of subtrees (such as the scale/range of the subtree) and simplify the subtrees based on it.

There have been some works that use the algebraic simplification [9] [10] [12] and numerical simplification [11] to regression and classification tasks. Specifically, in [9], the common algebraic simplification operators are applied during the GP process to simplify the GP trees, and the hashing technique was used to speed up the simplification process. In [10], a deep analysis was conducted on the effect of algebraic simplification on the GP building blocks, which discovered that although the simplification destroyed some building blocks, it can create new building blocks instead, which alleviate the concern of losing useful building blocks by simplification.

In [11], a numerical simplification method is proposed for GP. Specifically, during the fitness evaluation, it tracks the minimal and maximal values of each subtree. If the minimum and maximum of a subtree are close enough, then it replaces the subtree with a constant node in the middle of the range. Compared with the algebraic simplification, the numerical simplification cannot guarantee that the tree stays the same before and after the simplification. However, it has a higher capability of reducing the size of the GP tree.

The algebraic and numerical simplification methods were further analysed on classification tasks in [12], which showed that both simplification methods can greatly reduce the program size without sacrificing the classification performance.

The existing simplification methods for GP were only tested on classification and regression tasks, but it is unknown how they will perform on the GP for evolving dispatching rules for the DJSS problem. Unlike the classification/regression tasks, the dispatching rule learning for DJSS is not supervised (there is no target value for the priorities), and the number of features in DJSS and their scales can be different from those in classification and regression. In this paper, we aim to develop both generic and problem-specific algebraic simplification operators in the context of DJSS, and examine their efficacy.

## III. Designed Simplification Operators for Dynamic Job Shop Scheduling

The simplification operators used in this work can be divided into two groups. The first group consists of the generic algebraic operators that are independent of the problem domain knowledge. In other words, they can be applied to any problem domain. These operators are shown in Table I. In the table, the lowercase letters indicate the constants, while the capital letters stand for variables or features. The table considers the arithmetic operators $\{+, -, \times, \div\}$ and the $\min$, $\max$ and $if$ operators ($if(A, B, C) = B$ if $A > 0$, and $C$ otherwise), as these are in the function set of GP for evolving dispatching rules of DJSS.

TABLE I
The generic algebraic simplification operators.

| Precondition(s) | Simplification |
|---|---|
| $a + b$ | c, c = $a + b$ |
| $a - b$ | c, c = $a - b$ |
| $a \times b$ | c, c = $a \times b$ |
| $a \div b$ | c, c = $a \div b$ |
| $\min(a, b)$ | $a$ if $a < b$, $b$ otherwise. |
| $\max(a, b)$ | $b$ if $a < b$, $a$ otherwise. |
| $A - 0$ | $A$ |
| $A - A$ | $0$ |
| $A \div 1$ | $A$ |
| $A \div A$ | $1$ |
| $0 \div A$ | $0$ |
| $A + 0 = 0 + A$ | $A$ |
| $0 \times A = A \times 0$ | $0$ |
| $A \times 1 = 1 \times A$ | $A$ |
| $\min\{A, A\}$ | $A$ |
| $\max\{A, A\}$ | $A$ |
| $if(a, A, A)$ | $A$ |
| $if(a, A, B)$ and $a > 0$ | $A$ |
| $if(a, A, B)$ and $a \leq 0$ | $B$ |
| $\min\{A, A + a\}$ and $a \geq 0$ | $A$ |
| $\min\{A, A + a\}$ and $a < 0$ | $A + a$ |
| $\min\{A, A - a\}$ and $a > 0$ | $A - a$ |
| $\min\{A, A - a\}$ and $a \leq 0$ | $A$ |
| $\max\{A, A + a\}$ and $a > 0$ | $A + a$ |
| $\max\{A, A + a\}$ and $a \leq 0$ | $A$ |
| $\max\{A, A - a\}$ and $a \geq 0$ | $A$ |
| $\max\{A, A - a\}$ and $a < 0$ | $A - a$ |
| $\min\{A, \min\{A, B\}\}$ | $\min\{A, B\}$ |
| $\max\{A, \max\{A, B\}\}$ | $\max\{A, B\}$ |

The second group of simplification operators are more problem specific and need more problem domain knowledge. They are shown in Table II. The main difference between the operators in Table II and the corresponding ones in Table I is that the problem-specific operators in Table II consider the conditions on *variables/features* rather than constants. Since we have many terminals compared with constants, we expect that there will be much more feature nodes than constant nodes in the GP tree for DJSS. Thus, the operators in Table II may be more applicable than those in Table I during the evolutionary process.

On the other hand, the problem-specific operators in Table II require to estimate the sign or range of a feature or a subtree with feature(s), such as whether $B$ will be positive or not. We will make the estimation based on our domain knowledge and a recursive sign estimation algorithm.

### A. Sign Estimation

For estimating the sign of a subtree (e.g. $B$ in Table II), the designed recursive sign estimation algorithm first

TABLE II
THE PROBLEM-SPECIFIC SIMPLIFICATION OPERATORS.

| Precondition(s) | Simplification |
|---|---|
| $\min(A, A+B)$ and $B$ is **always non-negative** | $A$ |
| $\min(A, A+B)$ and $B$ is **always negative** | $A+B$ |
| $\min(A, A-B)$ and $B$ is **always non-positive** | $A$ |
| $\min(A, A-B)$ and $B$ is **always positive** | $A-B$ |
| $\max(A, A+B)$ and $B$ is **always non-positive** | $A$ |
| $\max(A, A+B)$ and $B$ is **always positive** | $A+B$ |
| $\max(A, A-B)$ and $B$ is **always non-negative** | $A$ |
| $\max(A, A-B)$ and $B$ is **always negative** | $A-B$ |

estimates the signs of the leaf nodes. Then, it recursively propagates the signs of child nodes to their parent, until the root node of the subtree is reached. The pseudo code of the recursive sign estimation algorithm is given in Algorithm 1. It includes two types of sign propagation, the `SignPropIf`$(\cdot)$ method for the "if" function that has three children, and the `SignProp`$(\cdot)$ method for all the other functions which have two children. The `SignPropIf`$(\cdot)$ and `SignProp`$(\cdot)$ methods set their outputs based on different situations of their inputs, as shown in Tables III and IV, respectively. In the tables, the notation # means that the sign can be either positive, negative, zero, or even unknown. For any other situation not satisfying any situation mentioned in the tables, the sign propagation method will return "UNKNOWN" as the output. Note that in Table IV, the functions "$+$", "$\times$", "min" and "max" are marked with $^*$, indicating that the situations can be swapped between $s1$ and $s2$, since the functions are symmetric.

**Algorithm 1:** `SignEst`$(B)$

**Input:** The subtree $B$ to be estimated the sign.
**Output:** The estimated sign
1 **if** $B$ *is a terminal* **then**
2 | **return** `sign`$(B)$;
3 **end**
4 **if** $root(B) = if$ **then**
5 | $s1 =$ `SignEst`$(child1)$;
6 | $s2 =$ `SignEst`$(child2)$;
7 | $s3 =$ `SignEst`$(child3)$;
8 | **return** `SignPropIf`$(s1, s2, s3)$;
9 **else**
10 | $s1 =$ `SignEst`$(lChild)$;
11 | $s2 =$ `SignEst`$(rChild)$;
12 | **return** `SignProp`$(root(B), s1, s2)$;
13 **end**

In line 2 of Algorithm 1, we determine the sign of a terminal based on our domain knowledge about the JSS problem. The detailed signs of the terminals will be given in Section V-A (Table VI).

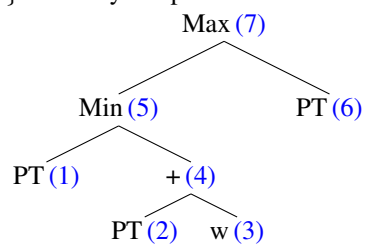
### B. Tree Simplification

A GP tree can be simplified multiple times at different subtree nodes. To make sure that the tree is fully simplified and no simplification operator is missed, we conduct the tree simplification in a recursive way from the leaf nodes. The recursive tree simplification is shown in Algorithm 2.

**Algorithm 2:** `TreeSimplify`$(T)$

**Input:** The tree $T$ to be simplified.
**Output:** The simplified tree.
1 **if** $T$ *is a terminal* **then**
2 | **return** $T$;
3 **end**
4 **foreach** *subtree* $S \in children(T)$ **do**
5 | $S' =$ `TreeSimplify`$(S)$;
6 | Replace $S$ with $S'$ in $T$;
7 **end**
8 Try the operators in Tables I and II to simplify $T$;
9 **return** $T$;

Below is an example tree that can be simplified. The numbers next to each node indicates the order that the simplification was (attempted) to be applied. In this example, the job weight $w$ is always positive from domain knowledge. Therefore, the first simplification occurs at position 5, where $\min\{\text{PT}, \text{PT}+\text{w}\}$ is simplified to PT. Then, at the root position 7, $\max\{\text{PT}, \text{PT}\}$ is finally simplified to PT.




## IV. GENETIC PROGRAMMING WITH ALGEBRAIC SIMPLIFICATION FOR DJSS

The flowchart of the GP with the proposed algebraic simplification operators for DJSS is shown in Fig. 1. It follows a typical GP process, except that at each generation, each individual is attempted to be simplified by Algorithm 2 after it is evaluated.

## V. EXPERIMENTAL STUDIES

### A. Experiment Design

To verify the effectiveness of the proposed GP with algebraic simplification, we consider the job shop scenarios with three different objectives, i.e. the mean tardiness (Eq. (1)), max tardiness (Eq. 2)), and mean weighted tardiness (Eq. (3)). We also consider two utilisation levels (0.85 and 0.95), which are the two commonly used settings to reflect the different degree

TABLE III
THE SIGN PROPAGATION METHOD FOR THE "IF" FUNCTION.

| $s1$ | $s2$ | $s3$ | output |
|---|---|---|---|
| $>0$ | # | # | $s2$ |
| $\leq 0$ | # | # | $s3$ |
| # | $>0$ | $>0$ | $>0$ |
| # | $\geq 0$ | $\geq 0$ | $\geq 0$ |
| # | $<0$ | $<0$ | $<0$ |
| # | $\leq 0$ | $\leq 0$ | $\leq 0$ |
| # | $=0$ | $=0$ | $=0$ |

TABLE IV
THE SIGN PROPAGATION METHOD FOR THE OTHER FUNCTIONS THAN "IF".

| $root(B)$ | $s1$ | $s2$ | output | $root(B)$ | $s1$ | $s2$ | output |
|---|---|---|---|---|---|---|---|
| $+^*$ | $>0$ | $\geq 0$ | $>0$ | $\max^*$ | $>0$ | # | $>0$ |
| $+^*$ | $\geq 0$ | $\geq 0$ | $\geq 0$ | $\max^*$ | $\geq 0$ | # | $\geq 0$ |
| $+^*$ | $=0$ | $=0$ | $=0$ | $\max^*$ | $<0$ | $<0$ | $<0$ |
| $+^*$ | $<0$ | $\leq 0$ | $<0$ | $\max^*$ | $\leq 0$ | $\leq 0$ | $\leq 0$ |
| $+^*$ | $\leq 0$ | $\leq 0$ | $\leq 0$ | $\max^*$ | $=0$ | $=0$ | $=0$ |
| $-$ | $>0$ | $\leq 0$ | $>0$ | $\min^*$ | $<0$ | # | $<0$ |
| $-$ | $\geq 0$ | $\leq 0$ | $\geq 0$ | $\min^*$ | $\leq 0$ | # | $\leq 0$ |
| $-$ | $=0$ | $=0$ | $=0$ | $\min^*$ | $>0$ | $>0$ | $>0$ |
| $-$ | $<0$ | $\geq 0$ | $<0$ | $\min^*$ | $\geq 0$ | $\geq 0$ | $\geq 0$ |
| $-$ | $\leq 0$ | $\geq 0$ | $\leq 0$ | $\min^*$ | $=0$ | $=0$ | $=0$ |
| $\times^*$ | $>0$ | $>0$ | $>0$ | $\div$ | $>0$ | $>0$ | $>0$ |
| $\times^*$ | $\geq 0$ | $\geq 0$ | $\geq 0$ | $\div$ | $\geq 0$ | $>0$ | $\geq 0$ |
| $\times^*$ | $=0$ | $=0$ | $=0$ | $\div$ | # | $=0$ | $>0$ (=1) |
| $\times^*$ | $<0$ | $<0$ | $>0$ | $\div$ | $<0$ | $<0$ | $>0$ |
| $\times^*$ | $\leq 0$ | $\leq 0$ | $\geq 0$ | $\div$ | $\leq 0$ | $<0$ | $\geq 0$ |
| $\times^*$ | $>0$ | $<0$ | $<0$ | $\div$ | $>0$ | $<0$ | $<0$ |
| $\times^*$ | $\geq 0$ | $\leq 0$ | $\leq 0$ | $\div$ | $<0$ | $>0$ | $<0$ |
| | | | | $\div$ | $\leq 0$ | $>0$ | $\leq 0$ |
| | | | | $\div$ | $\geq 0$ | $<0$ | $\leq 0$ |

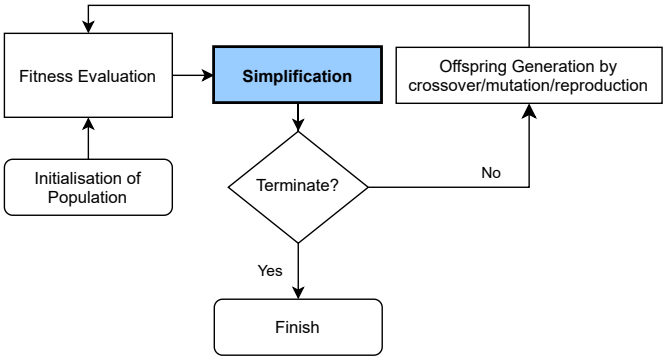



Fig. 1. Simplification during the Evolutionary Process

of busyness of the machines in the job shop. For example, a utilisation level of 0.85 indicates that the machines in the job shop are expected to by busy for 85% of the time. A higher utilisation level leads to a busier job shop with more challenging scheduling decisions. For all scenarios, we set the due date factor to be 4. Overall, we have six DJSS scenarios for experiments. The scenarios are denoted as $\langle obj, ulevel \rangle$.

Following the commonly used configuration [38] [39] [2], the DJSS simulation used in the experiments contains 10 machines, 5000 jobs, and 1000 warmup jobs. That is, starting from empty machines, the first 1000 arrived jobs are scheduled without recording. Then, the next 5000 jobs are scheduled and their completion times are recorded to calculate the objective values such as the mean tardiness. This is to avoid the initial warm-up stage with less busy schedules and estimate the steady-state performance of the scheduling. The jobs arrive following a Poisson process with an arrival rate $\lambda$ (set according to the utilisation level). The number of operations per job was randomly sampled from 2 to 10, and the processing time of each operation was sampled from 1 to 99.

During the training process, one simulation is used for the fitness evaluation at each generation, and the simulation is changed every generation to improve the generalisation. The test set includes 50 simulations unseen from the training.

A baseline GP without simplification (named BaselineGP) is taken into comparison to verify the effectiveness of the proposed simplification operators. For the sake of convenience, the newly proposed GP is named SimplificationGP. For each compared algorithm on each DJSS scenario, 30 independent runs were conducted, and the results were compared by statistical tests.

The parameter settings and the terminal set used by the GP algorithms are given in Tables V and VI. They are commonly used settings by existing studies [2] [38]. For the terminal set, we also give the sign of each terminal (or UNKNOWN if it can be either positive, negative or zero). It can be seen that many of the terminals are positive or non-negative, which can be potentially helpful to apply the simplification operators in Table II.

### B. Results and Discussions

Table VII shows the mean and standard deviation of the test performance of the compared GP algorithms for each of the six DJSS scenarios. The test performance is defined as below:

$$perf_{test}(rule) = \frac{1}{|\mathcal{I}_{test}|} \sum_{I \in \mathcal{I}_{test}} \frac{obj(rule, I)}{obj(rule_{ref}, I)}, \quad (4)$$

TABLE V
THE GP PARAMETER SETTINGS.

| Parameter | Value |
|---|---|
| Population Size | 1024 |
| Crossover Rate | 80% |
| Reproduction Rate | 5% |
| Number of Elites | 10 |
| Function Set | +, -, *, /, max, min, if |
| Maximum Tree Depth | 8 |
| Mutation Rate | 15% |
| Number Of Generations | 50 |

TABLE VI
THE TERMINALS USED BY GP AND THEIR SIGNS

| Notation | Description | Sign |
|---|---|---|
| NIQ | Number Of Operations in the Queue | $\geq 0$ |
| WIQ | Work in the Queue | $\geq 0$ |
| MWT | Machine Waiting Time | $\geq 0$ |
| PT | Processing Time of the Operation | $>0$ |
| NPT | Processing Time of the next Operation | $\geq 0$ |
| OWT | Operation Waiting Time | UNKNOWN |
| NWT | Next Waiting Time | UNKNOWN |
| WKR | Work Remaining | $\geq 0$ |
| NOR | Number of Operations remaining | $\geq 0$ |
| WINQ | Work in the next Queue | $\geq 0$ |
| NINQ | Number of Operations in the next Queue | $\geq 0$ |
| rFDD | Relative Flow Due Date | UNKNOWN |
| rDD | Relative Due Date | UNKNOWN |
| W | Job Weight | $>0$ |
| AT | Arrival Time | $>0$ |
| SL | Slack | UNKNOWN |

where $\mathcal{I}_{test}$ is the set of test simulations, $obj(rule, I)$ indicates the objective value (e.g. mean tardiness) of the schedule generated by $rule$ on the simulation $I$. $rule_{ref}$ is the reference rule for normalisation. We have also conducted the Wilcoxon rank sum test with significance level of 0.05 to compare the statistical significance between the results of the two algorithms. From Table VII, we can see that the propose SimplificationGP achieved very similar test performance to the BaselineGP. Its test performance was slightly better on the scenario ($\langle MWT, 0.85\rangle$), and slightly worse on two scenarios ($\langle Tmean, 0.95\rangle$, $\langle MWT, 0.95\rangle$). However, no statistical significance was found.

Table VIII shows the mean and standard deviation of the size of the final rule obtained by the 30 independent runs of the compared GP algorithms. From the table, we can see slight reductions on the rule size by SimplificationGP on four out of six scenarios ($\langle Tmean, 0.85\rangle$, $\langle MWT, 0.85\rangle$, $\langle Tmax, 0.95\rangle$ and $\langle MWT, 0.95\rangle$), although they are not statistically significant.

Figs. 2 and 3 show the curve of the test performance and size of the best rules obtained in each generation of the training process. From Fig. 2, we can see that there was no overfitting. The curves for the SimplificationGP are similar to those of the BaselineGP.

From Fig. 3, we can see that the size of the best rule was slightly smaller for four scenarios ($\langle Tmean, 0.85\rangle$, $\langle Tmax, 0.95\rangle$, $\langle MWT, 0.85\rangle$ and $\langle MWT, 0.95\rangle$), especially towards the end of the evolutionary process. In $\langle Tmax, 0.85\rangle$, the two curves were very similar. It is interesting to see that the size of the best rule obtained by SimplificationGP was obviously larger than that of BaselineGP in ($\langle Tmean, 0.95\rangle$). This is counter-intuitive, since the rule size should be reduced by the simplification. We will further analyse this pattern in the subsequent section.

Overall, we can see that with the proposed algebraic simplification operators, SimplificationGP managed to slightly reduce the rule size in a majority of the test scenarios without sacrificing the test performance.

TABLE VII
MEAN AND STANDARD DEVIATION OF THE TEST PERFORMANCE OBTAINED BY THE 30 INDEPENDENT RUNS OF THE COMPARED GP ALGORITHMS.

| Scenario | BaselineGP | SimplificationGP |
|---|---|---|
| $\langle Tmax, 0.85\rangle$ | 0.50(0.02) | 0.50(0.02) |
| $\langle Tmean, 0.85\rangle$ | 0.52(0.06) | 0.52(0.05) |
| $\langle MWT, 0.85\rangle$ | 0.62(0.13) | 0.61(0.12) |
| $\langle Tmax, 0.95\rangle$ | 0.72(0.02) | 0.72(0.03) |
| $\langle Tmean, 0.95\rangle$ | 0.68(0.02) | 0.69(0.02) |
| $\langle MWT, 0.95\rangle$ | 0.80(0.05) | 0.81(0.04) |

TABLE VIII
MEAN AND STANDARD DEVIATION OF THE RULE SIZE OBTAINED BY THE 30 INDEPENDENT RUNS OF THE COMPARED GP ALGORITHMS.

| Scenario | BaselineGP | SimplificationGP |
|---|---|---|
| $\langle Tmax, 0.85\rangle$ | 58.10(18.79) | 58.73(22.63) |
| $\langle Tmean, 0.85\rangle$ | 70.87(18.55) | 64.83(15.94) |
| $\langle MWT, 0.85\rangle$ | 67.77(19.53) | 64.33(17.30) |
| $\langle Tmax, 0.95\rangle$ | 67.30(22.06) | 63.63(22.37) |
| $\langle Tmean, 0.95\rangle$ | 70.87(20.45) | 75.73(23.79) |
| $\langle MWT, 0.95\rangle$ | 73.80(20.09) | 66.40(17.49) |

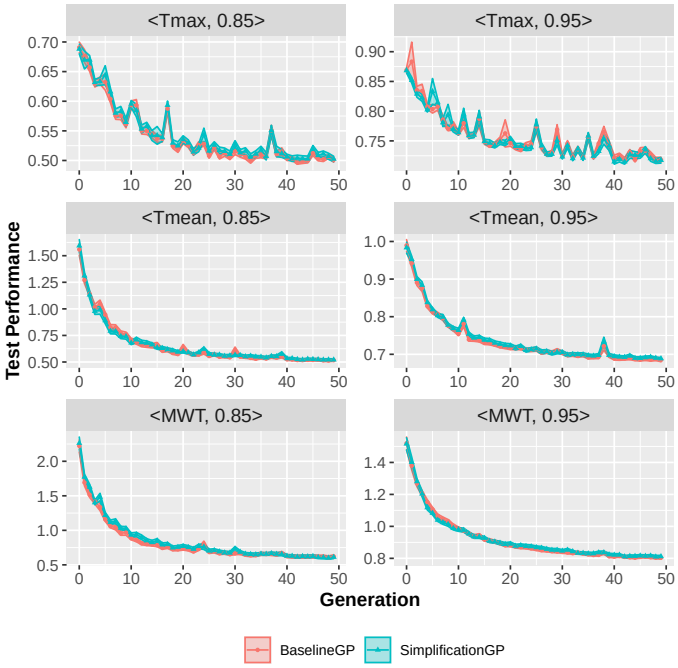



Fig. 2. The curves of the test performance of the compared algorithms.

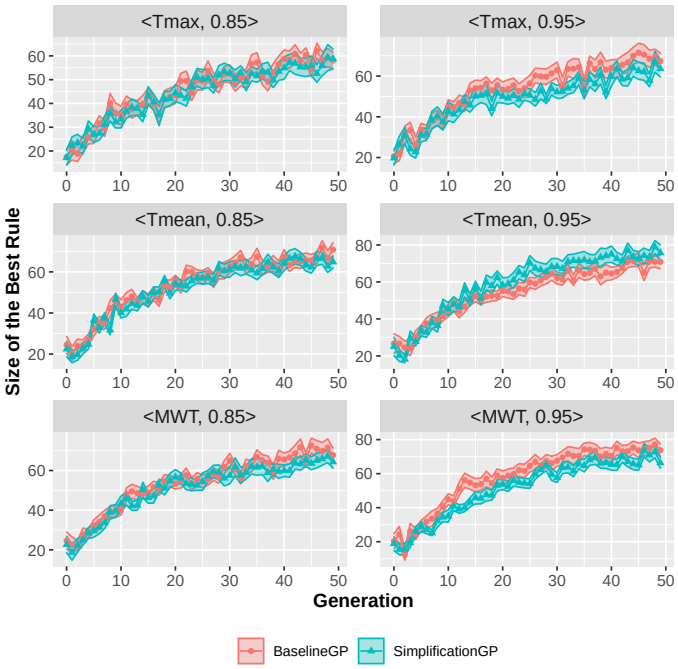



Fig. 3. The curves of the size of the best rule of the compared algorithms.

### C. Further Analysis

Compared with the patterns observed for the classification and regression tasks [9] [10] [12] [11], the advantage of using the algebraic simplification on reducing the rule size is not so obvious for the DJSS, as shown in Fig. 3. To gain a deeper understanding, we plot the curve of the *average size of all the rules in the population* during the training process, as shown in Fig. 4. From the figure, one can see that in most scenarios, SimplificationGP can reduce the average rule size more than the best rules size (e.g., $\langle Tmax, 0.85\rangle$, $\langle Tmax, 0.95\rangle$, $\langle MWT, 0.85\rangle$ and $\langle MWT, 0.95\rangle$). This indicates that simplifications were done on more trees that are not the best ones in the population.

For $\langle Tmean, 0.95\rangle$, the average size in the population of SimplificationGP was smaller than that of BaselineGP in the first 10 generations. However, after 10 generations, the size of SimplificationGP grew fast and exceeded that of BaselineGP. This may be because the best rule became large in SimplificationGP, and propagated its complex structure to other individuals in the population in the subsequent evolution.

Fig. 5 shows the number of simplified individuals in each generation of SimplificationGP. From the figure, we can see that the number of simplified individuals is relatively high (in the first generation) compared to the population size of 1024. In the first generation, over 300 individuals were simplified. However, after that, almost none of the individuals in the population could be simplified. This could partially explain why the rule size was not significantly improved during the GP process. From Fig. 4, we can see that even with over 300 individuals simplified in the first generation, the average size in the population was not much reduced. This indicates that the algebraic simplification can hardly simplify the individuals (e.g. mostly simplified only a 3-node subtree to a single node).

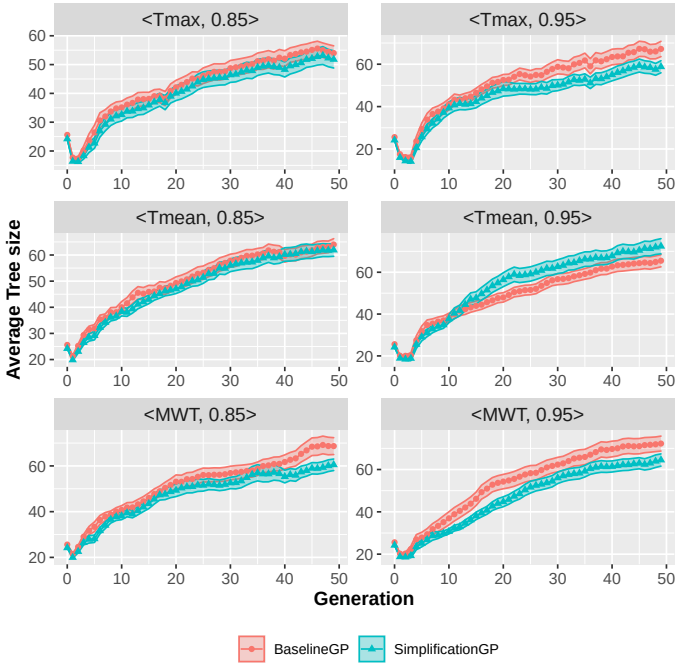



Fig. 4. Average size of the population of the compared algorithms.

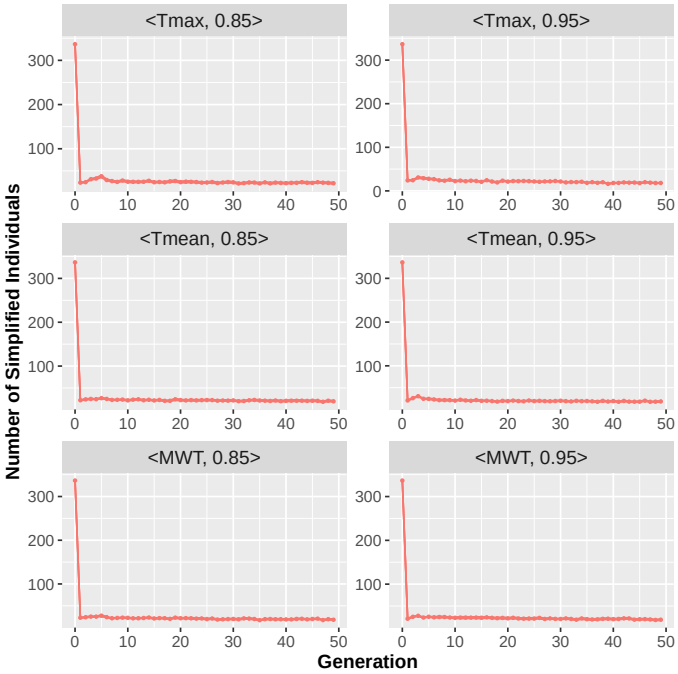



Fig. 5. The number of simplified individuals during the SimplificationGP process.

The reason that there are much fewer simplified individuals after the first generation may be that all possible cases for simplification are exhausted in the first generation. After the redundancies in the rules are simplified, new cases of redundancies tend to not occur.

## VI. Conclusions and Further Work

In this paper, we aim to reduce the size of the GP-evolved dispatching rules for DJSS while not worsening the effectiveness. This goal has been achieved by developing a range of algebraic simplification operators for the GP trees, including both problem independent and specific operators. We developed a new GP which simplifies the individuals using these algebraic simplification operators during the evolutionary process. The experimental results showed that with the algebraic simplification operators, we can evolve DJSS dispatching rules with slightly smaller size (number of nodes) and comparable test performance in most of the tested DJSS scenarios.

The further analysis showed the limitations of the proposed GP with algebraic simplification. First, it can only simplify individuals in the first generation. After that, it seems like the simplification situations have been exhausted, and the crossover and mutation can hardly generate new building blocks that can be simplified. Second, even if the simplification occurs, the individual cannot be simplified much (e.g., only one small subtree is simplified). This indicates that the pro-

posed algebraic simplification operators are not strong enough in the context of DJSS. The future directions could include developing more powerful simplification operators such as numerical simplification operators. We will also develop more intelligence schemes to decide which individuals to be simplified.

## REFERENCES


[1] S. Nguyen, Y. Mei, and M. Zhang, "Genetic programming for production scheduling: a survey with a unified framework," *Complex & Intelligent Systems*, vol. 3, no. 1, pp. 41–66, 2017.

[2] Y. Mei, S. Nguyen, and M. Zhang, "Constrained dimensionally aware genetic programming for evolving interpretable dispatching rules in dynamic job shop scheduling," in *Asia-Pacific Conference on Simulated Evolution and Learning*, pp. 435–447, Springer, 2017.

[3] D. Ouelhadj and S. Petrovic, "A survey of dynamic scheduling in manufacturing systems," *Journal of scheduling*, vol. 12, no. 4, pp. 417–431, 2009.

[4] H. Wang, Z. Jiang, Y. Wang, H. Zhang, and Y. Wang, "A two-stage optimization method for energy-saving flexible job-shop scheduling based on energy dynamic characterization," *Journal of Cleaner Production*, vol. 188, pp. 575–588, 2018.

[5] D. Gao, G.-G. Wang, and W. Pedrycz, "Solving fuzzy job-shop scheduling problem using de algorithm improved by a selection mechanism," *IEEE Transactions on Fuzzy Systems*, vol. 28, no. 12, pp. 3265–3275, 2020.

[6] H. Beaulieu, J. A. Ferland, B. Gendron, and P. Michelon, "A mathematical programming approach for scheduling physicians in the emergency room," *Health care management science*, vol. 3, no. 3, pp. 193–200, 2000.

[7] V. Sels, N. Gheysen, and M. Vanhoucke, "A comparison of priority rules for the job shop scheduling problem under different flow time-and tardiness-related objective functions," *International Journal of Production Research*, vol. 50, no. 15, pp. 4255–4270, 2012.

[8] J. Branke, S. Nguyen, C. W. Pickardt, and M. Zhang, "Automated design of production scheduling heuristics: A review," *IEEE Transactions on Evolutionary Computation*, vol. 20, no. 1, pp. 110–124, 2015.

[9] M. Zhang, P. Wong, and D. Qian, "Online program simplification in genetic programming," in *Asia-Pacific Conference on Simulated Evolution and Learning*, pp. 592–600, Springer, 2006.

[10] P. Wong and M. Zhang, "Effects of program simplification on simple building blocks in genetic programming," in *2007 IEEE Congress on Evolutionary Computation*, pp. 1570–1577, IEEE, 2007.

[11] D. Kinzett, M. Johnston, and M. Zhang, "Numerical simplification for bloat control and analysis of building blocks in genetic programming," *Evolutionary Intelligence*, vol. 2, no. 4, p. 151, 2009.

[12] D. Kinzett, M. Johnston, and M. Zhang, "How online simplification affects building blocks in genetic programming," in *Proceedings of the 11th Annual conference on Genetic and evolutionary computation*, pp. 979–986, 2009.

[13] P. Brucker, B. Jurisch, and B. Sievers, "A branch and bound algorithm for the job-shop scheduling problem," *Discrete applied mathematics*, vol. 49, no. 1-3, pp. 107–127, 1994.

[14] E. Mokotoff and P. Chrétienne, "A cutting plane algorithm for the unrelated parallel machine scheduling problem," *European Journal of Operational Research*, vol. 141, no. 3, pp. 515–525, 2002.

[15] P. J. Van Laarhoven, E. H. Aarts, and J. K. Lenstra, "Job shop scheduling by simulated annealing," *Operations research*, vol. 40, no. 1, pp. 113–125, 1992.

[16] M. Kolonko, "Some new results on simulated annealing applied to the job shop scheduling problem," *European journal of operational research*, vol. 113, no. 1, pp. 123–136, 1999.

[17] M. Dell'Amico and M. Trubian, "Applying tabu search to the job-shop scheduling problem," *Annals of Operations research*, vol. 41, no. 3, pp. 231–252, 1993.

[18] J. F. Gonçalves, J. J. de Magalhães Mendes, and M. G. Resende, "A hybrid genetic algorithm for the job shop scheduling problem," *European journal of operational research*, vol. 167, no. 1, pp. 77–95, 2005.

[19] F. Pezzella, G. Morganti, and G. Ciaschetti, "A genetic algorithm for the flexible job-shop scheduling problem," *Computers & Operations Research*, vol. 35, no. 10, pp. 3202–3212, 2008.

[20] P. D. Dominic, S. Kaliyamoorthy, and M. S. Kumar, "Efficient dispatching rules for dynamic job shop scheduling," *The International Journal of Advanced Manufacturing Technology*, vol. 24, no. 1, pp. 70–75, 2004.

[21] A. Kaban, Z. Othman, and D. Rohmah, "Comparison of dispatching rules in job-shop scheduling problem using simulation: a case study," *International Journal of Simulation Modelling*, vol. 11, no. 3, pp. 129–140, 2012.

[22] M. Jayamohan and C. Rajendran, "Development and analysis of cost-based dispatching rules for job shop scheduling," *European journal of operational research*, vol. 157, no. 2, pp. 307–321, 2004.

[23] O. Holthaus and C. Rajendran, "Efficient dispatching rules for scheduling in a job shop," *International Journal of Production Economics*, vol. 48, no. 1, pp. 87–105, 1997.

[24] E. K. Burke, M. R. Hyde, G. Kendall, G. Ochoa, E. Ozcan, and J. R. Woodward, "Exploring hyper-heuristic methodologies with genetic programming," *Computational intelligence*, pp. 177–201, 2009.

[25] H. Al-Sahaf, Y. Bi, Q. Chen, A. Lensen, Y. Mei, Y. Sun, B. Tran, B. Xue, and M. Zhang, "A survey on evolutionary machine learning," *Journal of the Royal Society of New Zealand*, vol. 49, no. 2, pp. 205–228, 2019.

[26] Y. Mei, S. Nguyen, B. Xue, and M. Zhang, "An efficient feature selection algorithm for evolving job shop scheduling rules with genetic programming," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 1, no. 5, pp. 339–353, 2017.

[27] F. Zhang, Y. Mei, and M. Zhang, "A two-stage genetic programming hyper-heuristic approach with feature selection for dynamic flexible job shop scheduling," in *Proceedings of the Genetic and Evolutionary Computation Conference*, pp. 347–355, 2019.

[28] T. Hildebrandt and J. Branke, "On using surrogates with genetic programming," *Evolutionary computation*, vol. 23, no. 3, pp. 343–367, 2015.

[29] F. Zhang, Y. Mei, and M. Zhang, "Surrogate-assisted genetic programming for dynamic flexible job shop scheduling," in *Australasian Joint Conference on Artificial Intelligence*, pp. 766–772, Springer, 2018.

[30] S. Nguyen, M. Zhang, and K. C. Tan, "Surrogate-assisted genetic programming with simplified models for automated design of dispatching rules," *IEEE transactions on cybernetics*, vol. 47, no. 9, pp. 2951–2965, 2016.

[31] F. Zhang, Y. Mei, S. Nguyen, and M. Zhang, "A preliminary approach to evolutionary multitasking for dynamic flexible job shop scheduling via genetic programming," in *Proceedings of the 2020 Genetic and Evolutionary Computation Conference Companion*, pp. 107–108, 2020.

[32] J. Park, Y. Mei, S. Nguyen, G. Chen, and M. Zhang, "Evolutionary multitask optimisation for dynamic job shop scheduling using niched genetic programming," in *Australasian Joint Conference on Artificial Intelligence*, pp. 739–751, Springer, 2018.

[33] E. K. Burke, M. Hyde, G. Kendall, and J. Woodward, "A genetic programming hyper-heuristic approach for evolving 2-d strip packing heuristics," *IEEE Transactions on Evolutionary Computation*, vol. 14, no. 6, pp. 942–958, 2010.

[34] Y. Liu, Y. Mei, M. Zhang, and Z. Zhang, "Automated heuristic design using genetic programming hyper-heuristic for uncertain capacitated arc routing problem," in *Proceedings of the Genetic and Evolutionary Computation Conference*, pp. 290–297, 2017.

[35] M. Bader-El-Den, R. Poli, and S. Fatima, "Evolving timetabling heuristics using a grammar-based genetic programming hyper-heuristic framework," *Memetic Computing*, vol. 1, no. 3, pp. 205–219, 2009.

[36] B. Tan, H. Ma, and Y. Mei, "A hybrid genetic programming hyper-heuristic approach for online two-level resource allocation in container-based clouds," in *2019 IEEE Congress on Evolutionary Computation (CEC)*, pp. 2681–2688, IEEE, 2019.

[37] B. Tan, H. Ma, Y. Mei, and M. Zhang, "A cooperative coevolution genetic programming hyper-heuristic approach for on-line resource allocation in container-based clouds," *IEEE Transactions on Cloud Computing*, 2020.

[38] S. Nguyen, M. Zhang, M. Johnston, and K. C. Tan, "Dynamic multi-objective job shop scheduling: A genetic programming approach," in *Automated scheduling and planning*, pp. 251–282, Springer, 2013.

[39] Y. Mei, S. Nguyen, and M. Zhang, "Evolving time-invariant dispatching rules in job shop scheduling with genetic programming," in *European Conference on Genetic Programming*, pp. 147–163, Springer, 2017.